

CENTRO PAULA SOUZA
CNWI-2A-M-MTEC INFORMÁTICA PARA INTERNET

APOLO
DAVI LUIS FERNANDES DOS SANTOS
LAURA
OTÁVIO SANCHES
PABLO GOMES

**CRIAÇÃO DE SITES EM NUVEM COM GITHUB:
DOCUMENTAÇÃO DO PROCESSO DE DESENVOLVIMENTO
E DOS COMANDOS UTILIZADOS**

São Paulo
2026

APOLO
DAVI LUIS FERNANDES DOS SANTOS
LAURA
OTÁVIO SANCHES
PABLO GOMES

**CRIAÇÃO DE SITES EM NUVEM COM GITHUB:
DOCUMENTAÇÃO DO PROCESSO DE DESENVOLVIMENTO
E DOS COMANDOS UTILIZADOS**

Trabalho apresentado ao curso Técnico em Informática para Internet do Centro Paula Souza, como requisito parcial de avaliação da disciplina, referente ao projeto de criação de sites em nuvem utilizando a plataforma GitHub.

Líder do grupo: Pablo Gomes

São Paulo

1 INTRODUÇÃO

Este documento registra o processo de construção do projeto de criação de sites em nuvem desenvolvido pelo nosso grupo, com o uso do Git e da plataforma GitHub. O objetivo aqui é deixar anotado tudo o que foi feito: como o repositório foi organizado, de que maneira cada integrante trabalhou na sua parte sem atrapalhar a dos outros, quais comandos foram digitados no terminal e por quê.

O tema escolhido pelo grupo foi jogos eletrônicos. Cada integrante ficou responsável por uma página dedicada a um jogo diferente, com liberdade para definir o próprio tema visual e a própria paleta de cores. Além dessas páginas, foi criada uma página inicial que funciona como ponto de partida, reunindo links para todas as outras e trazendo um texto sobre os jogos, seus benefícios e os mundos que eles constroem.

O repositório utilizado está disponível em: https://github.com/pablosGomes/sites_em_nuvem.2

ORGANIZAÇÃO DA EQUIPE

O grupo foi formado por cinco integrantes. Conforme combinado no início, cada pessoa cuidaria de uma página, e o líder ficaria responsável por documentar o processo e por juntar todo o material no final.

Durante o desenvolvimento houve um imprevisto: a Laura não conseguiu concluir a parte dela por motivos pessoais, e a página de The Last of Us acabou sendo desenvolvida pelo líder do grupo. Por isso, no histórico do repositório, o commit dessa página aparece com a autoria de pablosGomes, e não com a dela.

Quadro 1 – Divisão de tarefas entre os integrantes

Integrante	Jogo escolhido	Arquivo	Branch
Apolo	Batman: Arkham Knight	apolo.html	site_apolo
Davi	Subnautica	davi.html	site_davi
Laura	The Last of Us	laura.html	site_laura
Otávio	Minecraft	otavio.html	site_otavio
Pablo (líder)	Elden Ring	pablo.html	site_pablo
Pablo (líder)	Página inicial	index.html	main

3 FERRAMENTAS UTILIZADAS

Para o desenvolvimento do projeto foram utilizados os seguintes recursos:

- Git, para o controle de versão local, com todos os comandos digitados manualmente no terminal;
- GitHub, para hospedar o repositório remoto e permitir que os integrantes trabalhassem ao mesmo tempo;
- Visual Studio Code, como editor de código;
- HTML5, CSS3 e JavaScript, como linguagens base de todas as páginas;
- Bibliotecas externas carregadas por CDN em algumas páginas, como Tailwind CSS, GSAP e Lenis;
- Google Fonts, para as tipografias, e capturas oficiais dos jogos hospedadas na Steam, usadas apenas para fins ilustrativos.

Vale registrar que nenhuma interface gráfica foi usada para as operações de versionamento. Todos os merges, branches e envios foram feitos por linha de comando, como pedia o enunciado do trabalho.

4 CRIAÇÃO E CONFIGURAÇÃO DO REPOSITÓRIO

O repositório foi criado pela conta do líder diretamente no GitHub, já com o commit inicial contendo o arquivo README.md. Esse commit, identificado por c05c354, é a origem de todas as branches do projeto.

Cada integrante clonou o repositório na própria máquina e configurou o nome de usuário e o e-mail localmente, para que os commits saíssem identificados corretamente no histórico.

Quadro 2 – Comandos de clonagem e configuração inicial

```
git clone https://github.com/pablosGomes/sites_em_nuvem.git
cd sites_em_nuvem

git config user.name "seu-usuario"
git config user.email "seu-email@exemplo.com"

git status
git log --oneline
```

Fonte: elaborado pelos autores (2026).

5 TRABALHO COM BRANCHES

Conforme a orientação do trabalho, foi obrigatório abrir uma branch específica para cada página criada. Essa é a parte mais importante da organização do projeto: como cada integrante trabalha em uma branch separada, ninguém sobrescreve o arquivo do outro, e a branch principal só recebe o conteúdo quando a página já está pronta.

5.1 Criação das branches

Cada integrante criou a sua branch a partir da main e a enviou para o repositório remoto. O parâmetro `-b` cria a branch e já muda para ela na mesma operação, enquanto o `-u` define qual branch remota ela vai acompanhar.

Quadro 3 – Criação e envio de uma branch

```
git checkout -b site_pablo
git push -u origin site_pablo

# conferindo as branches existentes
git branch
git branch -a
```

Fonte: elaborado pelos autores (2026).

5.2 Ciclo de trabalho em cada página

Com a branch criada, o trabalho de cada integrante seguiu sempre o mesmo ciclo: editar os arquivos, conferir o que mudou, adicionar as alterações, gravar o commit com uma mensagem explicando o que foi feito e enviar para o GitHub.

Quadro 4 – Ciclo de alteração, commit e envio

```
git status
git diff

git add pablo.html pablo.css pablo.js
git commit -m "Pagina Elden Ring - Terras Intermedias"
git push origin site_pablo
```

Fonte: elaborado pelos autores (2026).

Para trazer atualizações que estivessem no repositório remoto, foram usados os comandos de busca e atualização:

Quadro 5 – Atualização a partir do repositório remoto

```
git fetch --all --prune
```

```
git pull origin main
```


6 INTEGRAÇÃO DAS BRANCHES NA MAIN

Depois que as páginas ficaram prontas, o líder trouxe cada branch para a main. Antes de qualquer merge foi feita uma verificação para saber se haveria conflito, comparando os arquivos alterados em cada branch com o ponto em comum entre elas.

Quadro 7 – Verificação prévia de conflitos

```
git merge-base main origin/site_laura
git diff --name-only <ancestral> origin/site_laura
```

Fonte: elaborado pelos autores (2026).

Como cada integrante mexeu apenas nos próprios arquivos, as listas não tinham nenhum nome em comum, e por isso os merges entraram sem conflito algum.

Os merges foram feitos com a opção `--no-ff`, que força a criação de um commit de merge mesmo quando o Git conseguiria apenas adiantar o ponteiro. Isso foi proposital: assim fica registrado no histórico o momento exato em que cada página entrou na main, o que ajuda na documentação exigida pelo trabalho.

Quadro 8 – Merge das branches na branch principal

```
git checkout main

git merge --no-ff origin/site_otavio -m "Merge da branch site_otavio na
main - pagina Minecraft (Otavio)"
git merge --no-ff origin/site_davi -m "Merge da branch site_davi na
main - pagina Subnautica 2 (Davi)"
git merge --no-ff origin/site_laura -m "Merge da branch site_laura na
main - pagina The Last of Us (Laura)"
git merge --no-ff origin/site_apolo -m "Merge da branch site_apolo na
main - pagina Batman Arkham Knight (Apolo)"

git push origin main
```

Fonte: elaborado pelos autores (2026).

Para conferir o resultado e enxergar o desenho das ramificações, foi usado o `log` em formato de grafo:

Quadro 9 – Conferência do histórico e da estrutura de branches

```
git log --oneline --graph --all
git ls-tree -r main --name-only
git branch -a --contains <hash>
```

Fonte: elaborado pelos autores (2026).

8 NAVEGAÇÃO ENTRE AS PÁGINAS

O enunciado pedia que as páginas se conectassem entre si, permitindo que o visitante navegasse com facilidade de uma para outra. Ao revisar o projeto depois dos merges, percebemos que três páginas não tinham nenhum link apontando para fora delas: quem entrasse nelas ficava sem saída.

A correção foi feita em duas frentes. Primeiro, cada página recebeu no rodapé uma lista com links para todas as outras. Depois, foi acrescentado um botão na barra de navegação de cada uma para voltar à página inicial, sempre com a estilização da própria página, para não quebrar a identidade visual que cada integrante montou.

A página inicial, por sua vez, apresenta as cinco páginas em cartões. Cada cartão tem dois links: um abre a página e o outro leva à branch correspondente no GitHub, o que deixa visível a estrutura de trabalho adotada pelo grupo.

9 REGISTRO DOS COMANDOS UTILIZADOS

O quadro a seguir reúne os comandos empregados ao longo do projeto, com uma explicação do que cada um faz. Todos foram executados manualmente no terminal.

Quadro 10 – Comandos Git utilizados no projeto

Comando	Finalidade
git clone <url>	Copia o repositório remoto para a máquina local.
git config user.name / user.email	Define a identidade que aparecerá nos commits.
git status	Mostra os arquivos alterados e ainda não gravados.
git diff	Exibe, linha a linha, o que mudou nos arquivos.
git branch	Lista as branches locais.
git branch -a	Lista também as branches do repositório remoto.
git checkout -b <nome>	Cria uma branch nova e muda para ela.
git checkout <branch>	Alterna entre branches já existentes.
git checkout <branch> -- <arquivo>	Traz um arquivo específico de outra branch.
git add <arquivos>	Prepara as alterações para o próximo commit.
git commit -m "mensagem"	Grava as alterações no histórico local.
git push origin <branch>	Envia os commits locais para o GitHub.
git push -u origin <branch>	Envia e vincula a branch local à remota.
git fetch --all --prune	Busca novidades do remoto e limpa referências antigas.
git pull origin main	Busca e já incorpora as alterações da main.
git merge --no-ff <branch>	Junta uma branch à atual criando commit de merge.
git merge-base <a> 	Mostra o commit em comum entre duas branches.
git log --oneline --graph --all	Exibe o histórico em forma de grafo.
git ls-tree -r <branch> --name-only	Lista os arquivos existentes em uma branch.
git tag <nome> <hash>	Marca um commit, servindo como ponto de retorno.
git reset --hard <hash>	Move a branch para um commit anterior.
git push --force-with-lease	Reescreve a branch remota com segurança.

Fonte: elaborado pelos autores (2026).

10 DIFICULDADES ENCONTRADAS

Nem tudo saiu como planejado, e vale registrar os problemas porque eles ensinaram bastante sobre como o Git funciona.

10.1 Branches que se afastaram

Em determinado momento a main local e a main do GitHub tinham commits diferentes uma da outra: na máquina havia os merges das páginas dos colegas, e no GitHub havia a página de Elden Ring, enviada em separado. As duas partiam do mesmo commit inicial, mas seguiram caminhos distintos.

A solução foi verificar antes se haveria conflito e, como os arquivos eram diferentes, unir as duas pontas com um merge comum. Nenhum conteúdo foi perdido.

10.2 Branch enviada com conteúdo indevido

A branch `site_pablo` havia sido criada acompanhando a main, e não uma branch remota própria. Por causa disso, ao ser enviada, ela subiu carregando todos os arquivos do projeto, e não apenas os da página de Elden Ring, ficando fora do padrão adotado pelas outras branches.

Antes de corrigir, foi criada uma marcação no commit atual, para garantir um ponto de retorno caso algo desse errado:

Quadro 11 – Marcação de segurança antes da correção

```
git tag backup-antes-do-rollback c79f3a0
git tag -l
```

Fonte: elaborado pelos autores (2026).

Em seguida a branch foi recriada a partir do commit inicial, recebendo somente os três arquivos da página, e depois reenviada:

Quadro 12 – Recriação da branch no padrão correto

```
git checkout -B site_pablo c05c354
git checkout main -- pablo.html pablo.css pablo.js
git commit -m "Pagina Elden Ring - Terras Intermedias"
git push --force-with-lease origin site_pablo
```

Fonte: elaborado pelos autores (2026).

Optou-se por `--force-with-lease` em vez de `--force` porque essa variação recusa a operação caso alguém tenha enviado algo para a branch enquanto isso, o que é uma proteção importante quando várias pessoas trabalham no mesmo repositório.

10.3 Conteúdo que não correspondia ao jogo

Durante a revisão final foi identificado que uma das páginas usava fotografias genéricas de bancos de imagem, legendadas como se fossem cenários do jogo, além de reproduzir a aparência de uma loja oficial, com botões de compra e a assinatura da empresa desenvolvedora no rodapé. A página foi refeita como material informativo, com capturas reais do jogo e um aviso deixando claro que se trata de trabalho escolar, sem vínculo com a empresa.

11 CONSIDERAÇÕES FINAIS

O projeto cumpriu o objetivo de exercitar o trabalho em equipe com controle de versão. A separação por branches se mostrou a decisão mais acertada: como cada integrante mexia apenas nos próprios arquivos e na própria branch, nenhum merge apresentou conflito, mesmo com cinco pessoas trabalhando ao mesmo tempo no repositório.

Também ficou claro na prática por que se recomenda cuidado com comandos que reescrevem histórico. O episódio da branch enviada de forma errada mostrou o valor de conferir a situação antes de agir e de criar um ponto de retorno, em vez de executar direto um comando destrutivo.

Ao final, o repositório reúne seis páginas conectadas entre si, cinco branches individuais preservadas e um histórico em que é possível acompanhar, commit por commit, como cada parte do site foi construída.

ago. 2026.